

Common SQL Data Types

INT, SMALLINT, BIGINT
Definition: Integer types with varying ranges.
Example: INT

VARCHAR(n)
Definition: Variable-length character string up to n chars. Example: VARCHAR(50)

CHAR(n)
Definition: Fixed-length character string of length n. Example: CHAR(10)

TEXT
Definition: Large variable-length text. Example: —

DATE, TIME, DATETIME, TIMESTAMP
Definition: Date and/or time values. Example: DATE

DECIMAL(p,s), NUMERIC(p,s)
Definition: Exact numeric with p total digits, s after decimal. Example: DECIMAL(10,2)

FLOAT, REAL, DOUBLE PRECISION
Definition: Approximate numeric types. Example: —

BOOLEAN
Definition: Logical TRUE/FALSE. Example: —

BLOB
Definition: Binary large object. Example: —

Aggregation & Grouping

COUNT(*), SUM(col), MIN(col), MAX(col), AVG(col)
Definition: Aggregate functions; ignore NULLs.

GROUP BY
Definition: Groups rows for aggregates. Example: GROUP BY col;

HAVING
Definition: Filters groups after aggregation. Example: HAVING AVG(x)>10;

ORDER BY
Definition: Sorts result set (ASC/DESC). Example: ORDER BY cnt DESC;

Eval Order
FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

Creating Tables

Customers / Products / Junction
Definition: Example of CREATE TABLE with PKs and FKs. Example:

```
CREATE TABLE Customers (
  CustomerID INT NOT NULL,
  FirstName VARCHAR(50),
  LastName VARCHAR(50),
  Email VARCHAR(100) UNIQUE,
  PRIMARY KEY (CustomerID)
);
CREATE TABLE Products (
  ProductID INT NOT NULL,
  ProductName VARCHAR(100),
  Price DECIMAL(10,2) DEFAULT 0.00,
  PRIMARY KEY (ProductID)
);
CREATE TABLE CustomerOrders (
  CustomerID INT NOT NULL,
  OrderID INT NOT NULL,
  OrderDate DATE NOT NULL,
  PRIMARY KEY (CustomerID, OrderID),
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),
  FOREIGN KEY (OrderID) REFERENCES Orders(OrderID)
);
```

Joins in SQL

Inner Join
Definition: Returns rows with matches in both tables. Example:

```
SELECT *
FROM A
JOIN B ON A.x = B.x;
```

Left Outer Join
Definition: All rows from left, matched or NULLs from right. Example: A LEFT JOIN B ON A.x=B.x;

Right Outer Join
Definition: All rows from right, matched or NULLs from left. Example: A RIGHT JOIN B ON A.x=B.x;

Full Outer Join
Definition: Rows matching in either, NULLs elsewhere. Example: A FULL JOIN B ON A.x=B.x;

Self Join
Definition: A table joined to itself via aliases. Example:

```
SELECT e1.Name AS Emp, e2.Name AS Mgr
FROM Employees e1
JOIN Employees e2
ON e1.ManagerID = e2.ID;
```

Data Modification

— update specific data with the WHERE clause

```
UPDATE table1 SET col1 = 1 WHERE col2 = 2
```

— insert values manually

```
INSERT INTO table1 (ID, FIRST_NAME, LAST_NAME)
VALUES (1, 'Rebel', 'Labs');
```

— or by using the results of a query

```
INSERT INTO table1 (ID, FIRST_NAME, LAST_NAME)
SELECT id, last_name, first_name FROM table2
```

Nested Queries

Correlated Subquery
Definition: Subquery evaluated per outer row. Example:

```
SELECT a.actorname,
       (SELECT m.genre
        FROM Movie m
        WHERE m.name = a.moviename)
AS genre
FROM ActedIN a;
```

Subquery in FROM
Definition: Treat subquery as table. Example:

```
SELECT x.name, x.rating
FROM (SELECT * FROM Movie WHERE
      rating>8) AS x
WHERE x.rating<9;
```

EXISTS / IN
Definition: Existential tests in WHERE. Example:

```
WHERE EXISTS (
  SELECT 1 FROM Movie m
  WHERE m.name=a.moviename
  AND m.genre='Sci-Fi');
-- OR --
WHERE a.moviename IN (
  SELECT m.name FROM Movie m
  WHERE m.genre='Sci-Fi');
```

ALL / NOT EXISTS
Definition: Universal tests. Example:

```
WHERE 100000<ALL(
  SELECT a.salary FROM ActedIN a
  WHERE a.moviename=m.name);
-- OR --
WHERE NOT EXISTS (
  SELECT 1 FROM ActedIN a
  WHERE a.moviename=m.name
  AND a.salary<100000);
```

Set Operations

UNION
Definition: Combines two queries, removes duplicates. Example: SELECT ... UNION SELECT ...;

UNION ALL
Definition: Combines two queries, keeps duplicates.

INTERSECT
Definition: Rows common to both, removes duplicates.

EXCEPT
Definition: Rows in first not in second.

Relational Algebra

$\sigma_C(R)$
Definition: Selection — picks rows satisfying C. Example: $\sigma_{year>2010}(Movie)$

$\pi_{A_1, \dots, A_n}(R)$
Definition: Projection — picks columns A_i . Example: $\pi_{name, genre}(Movie)$

$R_1 \cup R_2$
Definition: Union of two relations.

$R_1 \cap R_2$
Definition: Intersection of two relations.

$R_1 - R_2$
Definition: Set difference.

$R_1 \times R_2$
Definition: Cartesian product.

$\rho_{B_1, \dots, B_n}(R)$
Definition: Renaming relation/attributes. Example: $\rho_{M(name)}(Movie)$

$\gamma_{G: f(R)}$
Definition: Grouping + aggregate. Example: $\gamma_{genre: count(*)}(Movie)$

$\bowtie_C$
Definition: Theta-join = $\sigma_C(R_1 \times R_2)$. Example: see Joins box.

$\tau_{A_1, \dots, A_n}(R)$
Definition: Sorting by A_i . Example: $\tau_{year, name}(Movie)$

TEXT OPERATORS

Fetch names of cities that start with a 'P' or end with an 'S':

```
SELECT city
FROM cities
WHERE name LIKE 'P%';
-- OR --
SELECT city
FROM cities
WHERE name LIKE '%S';
```

DROP TABLE IF EXISTS WorksOn;

CREATE TABLE Project (pid INT PRIMARY KEY, name TEXT, startYear INT);

CREATE TABLE Developer (did INT PRIMARY KEY, name TEXT, hireYear INT);

CREATE TABLE WorksOn (pid INT REFERENCES Project(pid), did INT REFERENCES Developer(did), year INT);

1. Dev count per project-year Returns for each project and for each year the total number of developers who worked on that project in that year. Your query should return the project's ID, name, year, and number of developers, sorted by year in increasing order, and, within each year, sorted in decreasing order of the number of developers.

Solution: $\{E\}^+ \rightarrow \{B, C, E, F\}$ violates BCNF because it is neither trivial nor contains all attributes (i.e., E is not a superkey). So we need to decompose. We decompose into R1(B, C, E, F) and R2(A, D, E, G).

For R1(B, C, E, F), there are no more FD that violate BCNF, so we continue with R2(A, D, E, G). We find $\{G\}^+ = \{A, D\}$, so we decompose into R21(G, A, D) and R22(G, E). R22 has now two attributes, so it is necessarily in BCNF. For R21, there are no more FDs that violate BCNF.

The only closed sets are $\{A\}$, $\{A, B\}$, and $\{A, B, C, D\}$

The following set of FDs are:

$F = \{A \rightarrow B, B \rightarrow A, C \rightarrow ABD, D \rightarrow ABC\}$ - Decompose into the following relations:

$(A)^+ = \{A, B\}$ and $(B)^+ = \{A, B\}$.

$(AB)^+ = \{A, B\}$. This set is closed.

$(C)^+ = \{A, B, C, D\}$ and $(D)^+ = \{A, B, C, D\}$.

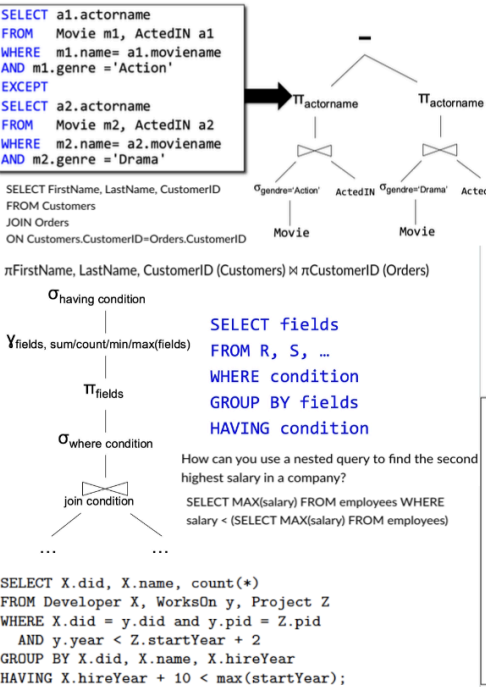
This set of FDs produces three closed sets.

SELECT P.pid, P.name, M.year, COUNT(*) AS cnt
FROM Project AS P
JOIN WorksOn AS W ON P.pid = W.pid
GROUP BY P.pid, P.name, W.year
ORDER BY W.year ASC, cnt DESC;

2. Inactive projects A project is called inactive if no developer worked on it since 2010 (including and after 2010). Write a SQL query to find all inactive projects. You should return their IDs and names.

-- Solution 1
SELECT P.pid, P.name
FROM Project AS P
WHERE NOT EXISTS (SELECT 1
FROM WorksOn AS W
WHERE W.pid = P.pid AND W.year >= 2010);

-- Solution 2
SELECT P.pid, P.name
FROM Project AS P
LEFT OUTER JOIN WorksOn AS W
ON P.pid = W.pid AND W.year >= 2010
GROUP BY P.pid, P.name
HAVING COUNT(W.pid) = 0;



Solution:

$\pi_{(X.did, X.name, C)}(\sigma_{(X.hireYear + 10 < M)}(\gamma_{(X.did, X.name, X.hireYear, count(*) > C, \max(Z.startYear)) \rightarrow M}(\sigma_{(Y.year < Z.startYear + 2)}(\rho_X(Developer) \bowtie_{(X.did=Y.did)} \rho_Y(WorksOn) \bowtie_{(Y.pid=Z.pid)} \rho_Z(Project)))))$

3. Top projects since 1990 For each year since 1990 (including and after 1990), return the project(s) that the most developers worked on during that year. Return the year and project ID. In case of a tie (if multiple projects had the maximum number of developers) return all of them.

-- Universal Quantifier Approach
SELECT W.year, W.pid
FROM WorksOn AS W
WHERE W.year >= 1990
GROUP BY W.year, W.pid
HAVING COUNT(*) >= ALL (SELECT COUNT(*)
FROM WorksOn AS W2
WHERE W2.year = W.year
GROUP BY W2.pid);

-- Witness Approach
WITH A AS (SELECT year, pid, count(*) AS cnt
FROM WorksOn
WHERE year >= 1990
GROUP BY year, pid),
B AS (SELECT D.did, D.name
FROM Project AS P
JOIN WorksOn AS W ON P.pid = W.pid
JOIN Developer AS D ON W.did = D.did
WHERE P.name = 'SystemX'
AND W.year BETWEEN P.startYear AND 2015
GROUP BY D.did, D.name, P.startYear
HAVING COUNT(*) = (2015 - P.startYear + 1);

4. SystemX devs SystemX is the oldest project of the company. Write a SQL query that returns all developers who worked every year on SystemX, from when it started until 2015. Your query should return the developers' ID and name.

-- Solution 1
SELECT D.did, D.name
FROM Project AS P
JOIN WorksOn AS W ON P.pid = W.pid
JOIN Developer AS D ON W.did = D.did
WHERE P.name = 'SystemX'
AND W.year BETWEEN P.startYear AND 2015
GROUP BY D.did, D.name, P.startYear
HAVING COUNT(*) = (2015 - P.startYear + 1);

A self-join is typically used to: Compare rows within the same table.
"In SQL, NONE values represent missing or unknown data." False
What does the SQL command LIMIT 10 do? Limits the result set to the first 10 rows.
Which of the following data integrity constraints ensures values in one table correspond to values in another table? Foreign key constraint.
Which of the following SQL clauses is best suited for filtering out duplicates based on specific columns, and not all columns? GROUP BY.
Which type of subquery is executed once for every row processed by the outer query? Correlated subquery.
What distinguishes a primary key from a foreign key? A primary key identifies a unique row in a table, while a foreign key links data between tables.
Is the following statement true or false? "SQL keywords are case-sensitive." False.
Which of the following allows you to uniquely identify a tuple? Super key.
What does a Common Table Expression (CTE) allow you to do? Define a temporary result set that can be referred to within a SELECT, INSERT, UPDATE, or DELETE statement.

The First Normal Form – 1NF

- A single cell must not hold more than one value (atomicity)
- There must be a primary key for identification
- No duplicated rows or columns
- Each column must have only one value for each row in the table

The Second Normal Form – 2NF

It's already in 1NF, Has no partial dependency: all non-key attributes are fully dependent on the primary key

(SELECT * FROM WorksOn Y WHERE X.did = Y.did AND Y.pid = Z.pid AND Y.year = 2015));

The Third Normal Form – 3NF

- It's in 2NF
- Has no transitive dependency: no non-prime attribute depends on another non-prime attribute

Solution:

Where A is

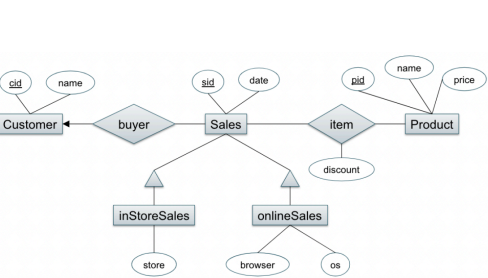
$\pi_{(X3.did)}(\rho_{X3}(Developer)) - \pi_{(X.did)}(A - B)$

Where B is

$\pi_{(Z.pid, X.did)}(\sigma_{(Y.year=2015)}(\rho_X(Developer) \bowtie_{(X.did=Y.did)} \rho_Y(WorksOn) \bowtie_{(Y.pid=Z.pid)} \rho_Z(Project)))$

SELECT FirstName, LastName, SUM(Quantity) FROM Customers JOIN Orders ON Customers.CustomerID=Orders.CustomerID JOIN OrderDetails ON Orders.OrderID=OrderDetails.OrderID GROUP BY FirstName, LastName

($\pi_{(FirstName, LastName, SUM(Quantity))}(\gamma_{(FirstName, LastName, SUM(Quantity))}(\pi_{(FirstName, LastName, Quantity)}(\pi_{(FirstName, LastName, CustomerID)}(\pi_{(CustomerID)} \bowtie \pi_{(CustomerID)}(\pi_{(OrderID)} \bowtie \pi_{(OrderID)}(\pi_{(OrderDetails)}))))$



R(A,B,C,D,E,F,G) with functional dependencies:

E → C
C → A
B → E
E → B
F → C
C → F

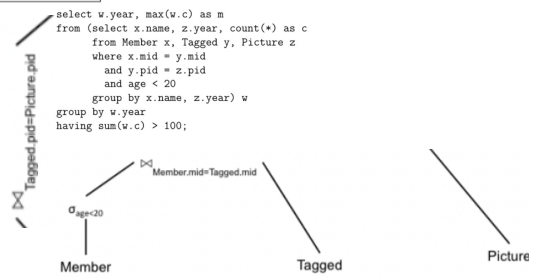
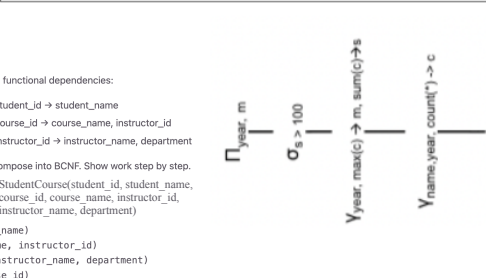
With functional dependencies:

- student_id → student_name
- course_id → course_name, instructor_id
- instructor_id → instructor_name, department

Decompose into BCNF. Show work step by step.

StudentCourse(student_id, student_name, course_id, course_name, instructor_id, instructor_name, department)

Student(student_id, student_name)
Course(course_id, course_name, instructor_id)
Instructor(instructor_id, instructor_name, department)
Enrollment(student_id, course_id)



Recursive Queries
A function $f: \text{NBN}$ is **computable** if there exists a systematic, mechanical procedure (an algorithm) that, given any input x , eventually halts and returns the correct output $f(x)$.

A **binary relation** R on a set X is a subset of $X \times X$.
A relation $R \subseteq X \times X$ is **transitive** if
 $(a, b) \in R \wedge (b, c) \in R \implies (a, c) \in R$.

The **transitive closure** of R is the smallest transitive relation R^+ containing R .
Equivalently, it contains all pairs of nodes (a, b) connected by some directed path.
Basic SQL has no mechanism to iterate until fixpoint, so it cannot express transitive closure or similar recursive queries.

Recursive Common Table Expressions (CTEs)

Modern SQL adds WITH RECURSIVE to define a fixpoint:

```
WITH RECURSIVE t AS (  
  -- Base case:  
  SELECT * FROM G  
  UNION  
  -- Recursive case:  
  SELECT x.a, y.b  
  FROM G x, t y  
  WHERE x.b = y.a  
)  
SELECT * FROM t;
```

Base case: initializes t with direct edges. **Recursive case:** joins new with previous to add indirect paths. Termination occurs when no new rows can be added.
On table Flights(origin, destination), find all cities reachable from San Diego:

```
WITH RECURSIVE  
  ReachableCities(city)  
AS (  
  SELECT destination  
  FROM Flights  
  WHERE origin = 'San Diego'  
  UNION ALL  
  SELECT f.destination  
  FROM Flights f  
  JOIN ReachableCities rc  
  ON f.origin = rc.city  
)  
SELECT city FROM  
  ReachableCities;
```

This collects each new destination until no more appear.

Given employees(employee_id, employee_name, manager_id), build the full reporting tree:

```
WITH RECURSIVE  
  employee_hierarchy AS (  
  -- Top-level managers:  
  SELECT employee_id,  
  employee_name,  
  manager_id  
  FROM employees  
  WHERE manager_id IS NULL  
  UNION ALL  
  -- Add direct reports  
  iteratively:  
  SELECT e.employee_id,  
  e.employee_name,  
  e.manager_id  
  FROM employees e  
  JOIN employee_hierarchy eh  
  ON e.manager_id = eh.employee_id  
)  
SELECT * FROM  
  employee_hierarchy;
```

Starts from root managers and pulls in all descendants.

Non-terminating Recursion This view never reaches a fixpoint unless externally limited:

```
CREATE RECURSIVE VIEW t AS  
  SELECT 0 AS N  
  FROM r  
  UNION  
  SELECT N+1 AS N  
  FROM t;
```

Each iteration increments N forever.

Query Evaluation
Query evaluation is the process of executing a query to retrieve data from a database. Query evaluation proceeds in three main phases:

- **Parsing:** Analyze the query's syntax and convert it into an internal representation (e.g. a parse tree).

- **Optimization:** Generate a query plan that minimizes resource usage and execution time, selecting both logical

(relational-algebra) and physical operators.
- **Execution:** Retrieve data from disk or memory and process it according to the chosen plan.

Physical Algorithms Each logical operator can be implemented by multiple algorithms, for example:

- Main-memory hash join
- External-memory merge join
- Partitioned hash join
- Sort-based grouping

Main-Memory Join Algorithms For the relational-algebra expression $\text{Movie} \bowtie_{\text{Movie.mid}=\text{ActedIn.mid}} \text{ActedIn}$, when both tables fit in main memory, possible implementations are:

- **Nested-loop join:** $O(n^2)$.
- **Merge join:** $O(n \log n)$ if inputs are pre-sorted.
- **Hash join:** $O(n)$ on average, worst-case $O(n^2)$.

Materialized evaluation computes an operator's entire output, stores it in a temporary relation, then feeds it to the next operator. Pros: allows reuse; cons: additional I/O and storage.

Pipelined evaluation streams tuples directly from one operator to the next without fully materializing intermediate results, reducing I/O and memory usage but limiting reuse.

A **physical plan** must specify:

- The algorithm for each operator (e.g. nested-loop vs. hash join).
- Whether to pipeline or materialize between operators.
- The access path for each base relation (full scan vs. index scan).

Data Storage and Indexing
CPU registers are the fastest storage (0.25–0.5 ns), smallest capacity (bytes–KB), volatile, and most expensive per bit.

Cache (SRAM) is organized in L1/L2/L3 levels (0.5–25 ns), volatile, stores frequently accessed data to reduce main-memory accesses.

DRAM (4 GB–32 GB) with access times 80–250 ns, volatile, balances cost and capacity.

HDDs/SSDs (hundreds of GB–TB), persistent, slowest access (μs –ms), most cost-effective per bit. A disk consists of platters, tracks (concentric circles), sectors (smallest addressable units), a read/write head on an actuator arm, and a spindle motor.

Rotational Latency Time for the disk to rotate the desired sector under the head; inversely proportional to RPM. Data is transferred in blocks (pages); sequential block reads are far more efficient than random reads.

Heap File An unsorted file of records; insertion is fast but queries require scanning.

Sequential File Records sorted by a key, enabling binary search and efficient range queries.

Index An auxiliary file mapping search keys to record pointers. Entries are (key, pointer).

Index Types - **Hash Index:** Fast for point queries. - **B+-Tree Index:** Supports both point and range queries. - **Clustered vs. Unclustered:** Clustered index dictates physical order (one per table); unclustered does not.

- **Primary vs. Secondary:** Primary covers the primary key, secondary covers other attributes.

Creating Indexes

```
CREATE INDEX idx_V_N ON  
  V(N);  
CREATE INDEX idx_V_P_M ON  
  V(P, M);
```

Index idx.V.N speeds up WHERE $N = ?$; composite index idx.V.P.M speeds up WHERE $P = ?$ AND $M = ?$.

Index Selection Guidelines
Index attributes appearing in WHERE (equality or range) or join conditions. Use composite indexes when multiple columns frequently appear together. Cluster indexes for range-heavy access (>1–2% of tuples). Weigh index maintenance overhead against query performance gains.

Database Design Process - Conceptual Model: High-level entities and relationships (e.g. Company makes Product)

- **Relational Model:** Tables with attributes and integrity constraints.

- **Normalization:** Decomposition to eliminate anomalies (redundancy, update, deletion).

- **Physical Schema:** File organization and storage structures.

Attribute Closure For a set of attributes X in schema R with FD set $\mathcal{F}$, the **closure** X^+ is all attributes functionally determined by X . Compute X^+ by: $X^+ \leftarrow X$; repeat while X^+ grows: for each $Y \rightarrow Z \in \mathcal{F}$ such that $Y \subseteq X^+$, set $X^+ \leftarrow X^+ \cup Z$.

Let $R(A, B, C, D, E, F)$ with FDs: $A, B \rightarrow C$, $A, D \rightarrow E$, $B \rightarrow D$, $A, F \rightarrow B$.
 $\{A, B\}^+ = \{A, B, C, D, E\}$,
 $\{A, F\}^+ = \{A, F, B, C, D, E\}$

Superkey and Key A **superkey** is $X \subseteq R$ such that X^+ contains all attributes of R . A **key** is a minimal superkey (no proper subset is a superkey).

An FD $X \rightarrow A$ is **safe** only if X is a superkey; otherwise decompose the relation on FDs to achieve a normal form free of anomalies.

Entity-Relationship Diagrams
An **entity set** is a collection of similar real-world objects, e.g. Patient, Procedure, or Doctor.

An **entity** is an individual object in an entity set, such as a particular patient or a specific medical procedure.

An **attribute** is a property of an entity or relationship, for example, a patient's PatientID, Name, or DateOfBirth.

A **relationship** is an association among two or more entity sets, such as “performs” between Doctor and Procedure.

A **key** for an entity set is an attribute (or minimal combination of attributes) that uniquely identifies each entity in that set, e.g. PatientID for Patient.

Entity set $\rightarrow$ a table with columns for each attribute and a primary key.

One-One or One-Many: add a foreign key in the “many” side.

Many-Many: create a junction table with foreign keys to each participating entity.

Multi-way: create a table with foreign keys to all involved entities plus relationship attributes.

Use ISA to define subclasses (specialized entity sets).

A **weak entity set** has no key of its own; its key is formed by combining a discriminator with the primary key of its owning (identifying) entity set.

Boyce-Codd Normal Form (BCNF)

Anomalies: Redundancy in table records can lead to insertion, deletion, and update anomalies.

Decomposition: We remove anomalies by decomposing a table into smaller tables that preserve the original data.

Functional Dependencies (FDs): FDs capture how attributes determine one another and guide decomposition.

BCNF: A strong normal form that ensures no “bad” FDs remain, thereby eliminating anomalies. A relation schema R is in **Boyce-Codd Normal Form** if for every non-trivial functional dependency $X \rightarrow Y$ holding on R , the set of attributes X is a superkey of R . Here “non-trivial” means $Y \not\subseteq X$.

Equivalently, for every FD $X \rightarrow Y$ in R either - the FD is trivial ($Y \subseteq X$), or - the closure X^+ (under all FDs of R) equals the full set of attributes of R .

A **trivial dependency** is one where the RHS of the dependency is a subset of the LHS. A **non-trivial dependency**, on the other hand, is where the RHS is not a subset of the LHS.

To test $R = S_1 \bowtie \dots \bowtie S_k$, one constructs a **chase tableau** and repeatedly applies the FDs to equate symbols. If at any point a row becomes identical to the “identity” row, the join is lossless.

Chase for $R(A, B, C, D)$ with $A \rightarrow B$, $B \rightarrow C$, $CD \rightarrow A$

Decompose into $S_1(A, D)$, $S_2(A, C)$, $S_3(B, C)$.

Normal Forms
- **First Normal Form (1NF):** Every attribute contains only atomic (indivisible) values; no repeating groups or arrays.

- **Second Normal Form (2NF):** A relation is in 1NF and every non-prime attribute is fully functionally dependent on the whole of every candidate key (i.e., no partial dependencies).

- **Third Normal Form (3NF):** A relation is in 2NF, and every non-prime attribute is non-transitively dependent on every candidate key. More formally: for every FD $X \rightarrow A$, either X is a superkey, or A is a prime attribute.

- **Boyce-Codd Normal Form (BCNF):** A stricter version of 3NF. For every non-trivial FD $X \rightarrow A$, X must be a superkey.

BCNF eliminates all redundancy due to functional dependencies but may require decomposition that does not preserve all original FDs. 3NF ensures dependency preservation and lossless decomposition, potentially at the cost of allowing some controlled redundancy.

```
SELECT e.courseName  
FROM Enrollment AS e JOIN  
  Student AS s ON e.s_id = s.sid  
WHERE e.yearTaken > 2020  
ORDER BY s.yearEnrolled;
```

Create a clustered index for this query, also state the type of index:

```
CREATE CLUSTERED INDEX  
  idx\_q1 ON  
  Student(yearEnrolled);
```

You could cluster on either: $s.\text{yearEnrolled} \rightarrow$ optimizes the ORDER BY or $e.\text{yearTaken} \rightarrow$ optimizes the WHERE clause

Sorting is expensive as compared to filtering, so optimizing ORDER BY would be preferred.

B+ Tree Internal nodes store keys and pointers to child, Leaf nodes store keys and actual data records. All keys in the internal nodes are sorted in ascending order. Each internal node has one more child pointer than the number of keys it stores. All data

records are stored in the leaf nodes, and they are linked together to form a chain.

Consider the following database about picture tagging

Website: Member(mid, name, age)

Picture(pid, year)

Tagged(mid, pid)

Relation Member contains a tuple for each registered member of the Website; Picture contains the metadata of the pictures (its key pid, and the year when the picture was taken), and Tagged says which user was tagged in what picture. Primary keys are underlined. Attributes values may be null.

Write a SQL query that retrieves the names of all members that were tagged both in the year 2011 and in 2014. Return them sorted in lexicographic order.

```
select x.name  
from Member x, Tagged y1,  
  Picture z1, Tagged y2,  
  Picture z2  
where x.mid = y1.mid and  
  y1.pid = z1.pid and  
  z1.year = 2011  
  and x.mid = y2.mid and  
  y2.pid = z2.pid and  
  z2.year = 2014  
order by x.name;
```

Write a SQL query that returns for each member the total number of times that member was tagged in 2015. Order the results by the members' names. You should include all members, even those who were not tagged in that year.

```
select x.name, count(z.pid)  
from Member x left outer join  
  Tagged y on x.mid = y.mid  
  left outer join  
  Picture z on y.pid = z.pid and z.year = 2015  
group by x.mid, x.name  
order by x.name;
```

Write a SQL query that returns the mid's and names of all members that were tagged only in pictures were Alice was also tagged.

```
select x.mid, x.name  
from Member x  
where x.mid not in  
  (select y.mid  
  from Tagged y  
  where y.pid not in  
    (select v.pid from  
      Member u, Tagged v  
      where u.name = 'Alice'  
      and u.mid = v.mid));
```

- Sales represents individual sales. One sale contains several products bought by one single customer.

inStoreSales consists of the sales in brick-and-mortar stores; each such sale contains the store name. onlineSales consists of online sales; each such sale includes the name of the browser and the operating system used during the purchase. cid, sid, pid are int, price and discount are float, the rest are text. No attributes may be null, except for inStoreSales.store and onlineSales.os

```
drop table if exists Item,  
  onlineSales,  
  inStoreSales, Sales,  
  Product, Customer;
```

```
create table Customer (  
  cid int primary key,  
  name text not null);  
create table Product (  
  pid int primary key,  
  name text not null,  
  price float not null);  
create table Sales (  
  sid int primary key,  
  date text not null,  
  cid int not null references  
    Customer);  
create table inStoreSales (  
  sid int primary key  
  references Sales,  
  store text);  
create table onlineSales (  
  sid int primary key  
  references Sales,  
  browser text not null,  
  os text);  
create table Item (  
  pid int references Product,  
  sid int references Sales,  
  discount float not null,  
  primary key (pid, sid));
```

$Q_{better} = \pi_{sid}(\sigma_{grade > pgrade}(\text{Enroll} \bowtie Prereq \bowtie Ppid, sid, pgrade(\pi_{cid, sid, grade}(\text{Enroll}))))$

$Q = \pi_{name}(\text{Student} \bowtie Q_{better})$